

ESP32+Docker+API REST

Chaves Laura

Cundinamarca, Universidad de Cundinamarca

Fusagasugá, Colombia

lcchaves@ucundinamarca.edu.co

I. INTRODUCCIÓN

Este documento resume en gran medida el laboratorio de conexión Wifi + MQTT + Base de datos para conectarla con una página web tipo dashboard. En pocas palabras, el objetivo del laboratorio era enviar datos de sensores hacia una gráfica visual que ilustre los datos en tiempo real de lo que se está recibiendo en el sensor. Se describen las herramientas utilizadas y el procedimiento paso a paso, exceptuando los pasos que se han llevado a cabo en los laboratorios anteriores, como el diagrama de conexiones, las propias conexiones y el código en Arduino IDE. Con este proyecto, se espera reforzar los conocimientos adquiridos de dichos laboratorios y adquirir nuevos con el presente.

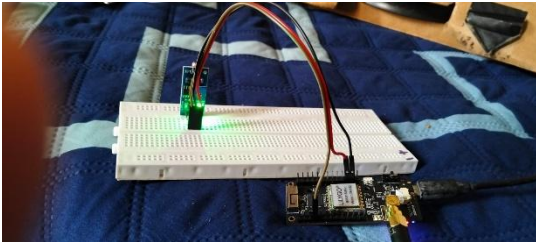
II. HERRAMIENTAS UTILIZADAS

Para el desarrollo del proyecto se emplearon las siguientes herramientas:

- Docker y Docker Compose: plataforma de contenedores utilizada para orquestar todos los servicios del sistema.
- EMQX: broker MQTT encargado de recibir y distribuir los mensajes publicados por el sensor.
- MariaDB: motor de base de datos relacional donde se persisten las lecturas recibidas del sensor.
- phpMyAdmin: interfaz gráfica para la administración y consulta de la base de datos MariaDB.
- Python 3.12 con Flask y paho-mqtt: backend del sistema. Flask expone el endpoint de datos al frontend, mientras que paho-mqtt gestiona la suscripción al broker EMQX.
- Nginx: servidor web que sirve el dashboard estático al navegador y actúa como proxy inverso hacia los demás servicios.
- Chart.js: librería de JavaScript utilizada en el frontend para actualizar la gráfica de lecturas en tiempo real.

A. Subtítulos

III. PROCESO DE CONSTRUCCION E IMPLEMENTACIÓN



A. Configuración del Entorno

Se definió el archivo .env con las credenciales de MariaDB y los parámetros de EMQX. Este archivo es referenciado por Docker Compose para inyectar las variables de entorno en cada contenedor, evitando escribir datos sensibles directamente en el código.

B. Operación con Docker Compose

Se configuró el archivo docker-compose.yml con cinco servicios: emqx, mariadb, phpmyadmin, backend y nginx. Cada servicio se comunica a través de una red interna virtual llamada app, que facilita la comunicación entre los contenedores. Los servicios que dependen de MariaDB utilizan la condición service_healthy para garantizar que la base de datos esté disponible antes de intentar conectarse. A continuación, se describe superficialmente el papel que cumple cada servicio dentro del sistema:

- Emqx: Broker de protocolo MQTT a donde llegarán los datos provenientes de la ESP32 para que luego el sistema los intercepte.
- mariaDB: El host de base de datos, seleccionado gracias a su practicidad y licencia de Software Libre.
- phpMyAdmin: El gestor de bases de datos, seleccionado gracias a lo sencillo que resulta acceder a él y porque el usuario y contraseña puede establecerse para que no requiera inicio de sesión al entrar.
- Backend: se construye una imagen desde 0 a partir del Dockerfile que se encuentra en la carpeta Backend del código. De hecho, dicho Dockerfile contiene la imagen de Python 3.12 para usarse en app.py.
- Nginx: toma todos los puertos y los esconde, dejando accesible únicamente el puerto directo a la página web (80), por lo que para acceder al resto de servicios, se utilizan URLs personalizadas.

```
1 services:
2
3   emqx:
4     image: emqx/emqx:latest
5     restart: unless-stopped
6     ports:
7       - "1883:1883"
8       - "4444:4444"
9       - "1801:1801"
10    networks: [app]
11    healthcheck:
12      test: ["CMD", "emqx", "ping"]
13      interval: 30s
14      timeout: 30s
15      retries: 5
16
17   mariadb:
18     image: mariadb:10.5
19     restart: unless-stopped
20     environment:
21       - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
22       - MYSQL_DATABASE=${MYSQL_DATABASE}
23       - MYSQL_USER=${MYSQL_USER}
24       - MYSQL_PASSWORD=${MYSQL_PASSWORD}
25     volumes:
26       - mariadb_data:/var/lib/mysql
27     networks: [app]
28     healthcheck:
29       test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
30       interval: 30s
31       timeout: 30s
32       retries: 5
33
34   phpmyadmin:
35     image: phpmyadmin:latest
36     restart: unless-stopped
37     depends_on:
```

C. Desarrollo del backend en Python

Se desarrolló el archivo app.py, el cual cumple dos funciones simultáneas. En un hilo secundario, se suscribe al topic test01 del broker EMQX mediante paho-mqtt y, ante cada mensaje recibido, inserta el valor en la tabla lecturas de MariaDB. En el hilo principal, Flask expone el endpoint /api/data, que retorna las últimas 60 lecturas en formato JSON ordenadas cronológicamente.

Al iniciar, el backend verifica si la tabla lecturas existe y la crea en caso contrario, lo que elimina la necesidad de ejecutar scripts SQL de forma manual.

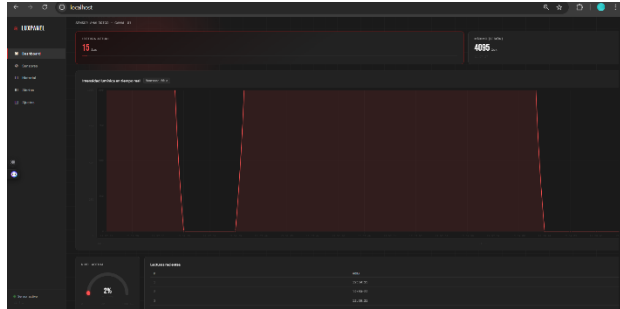
ID	Nombre de Base	Fecha
1	100	2026-05-05 19:40:09
2	350	2026-05-05 19:40:32
3	0	2026-05-06 17:55:47
4	6	2026-05-06 17:55:49
5	6	2026-05-06 17:55:51
6	2	2026-05-06 17:55:53
7	6	2026-05-06 17:55:55
8	5	2026-05-06 17:55:57
9	6	2026-05-06 17:55:59
10	7	2026-05-06 17:56:01
11	5	2026-05-06 17:56:02
12	7	2026-05-06 17:56:04
13	4	2026-05-06 17:56:07
14	5	2026-05-06 17:56:09
15	3	2026-05-06 17:56:10
16	2	2026-05-06 17:56:13
17	3	2026-05-06 17:56:15
18	7	2026-05-06 17:56:17
19	5	2026-05-06 17:56:19
20	0	2026-05-06 17:56:21
21	6	2026-05-06 17:56:23
22	15	2026-05-06 17:56:25
23	5	2026-05-06 17:56:27
24	6	2026-05-06 17:56:29
25	5	2026-05-06 17:56:31

D. Configuración de Nginx

Se configuró Nginx como punto de entrada único del sistema. Las peticiones a / sirven el dashboard estático, las peticiones a /api/ se redirigen al backend en Python, las peticiones a /pma/ se redirigen a phpMyAdmin y las peticiones a /emqx/ se redirigen al panel de administración del broker.

E. Desarrollo del Dashboard

El frontend consume el endpoint /api/data cada tres segundos mediante la función fetch de JavaScript. Con los datos recibidos, actualiza la gráfica de Chart.js, los indicadores de valor actual, máximo, mínimo y promedio, el gauge circular y la tabla de lecturas recientes.



F. Despliegue y Verificación

Una vez completos los archivos, se ejecutó el comando docker compose up --build -d para construir las imágenes y levantar todos los contenedores. La verificación del flujo completo se realizó publicando un mensaje de prueba en el topic test01 desde el EMQX y comprobando que el valor aparecía en el dashboard en menos de tres segundos.

Name	Container ID	Image	Port(s)	CPU (%)	Mem	Actions
testserverdoc	-	-	-	0%	0B / 0B	[Stop] [Refresh] [Logs]
testserverdoc	-	-	-	0%	0B / 0B	[Stop] [Refresh] [Logs]
testserverdoc	-	-	-	10.56%	589.2K	[Stop] [Refresh] [Logs]
mariaadb-1	3a554cca2900	mariaadb:10	-	1.09%	127.4K	[Stop] [Refresh] [Logs]
emqx-1	78c2ba95520e	emqx/emqx:4.10.0	18083-18083 (T)	7.22%	404.8K	[Stop] [Refresh] [Logs]
phpmyadmin	5d8d520e0454	phpmyadmin	-	0%	13.73K	[Stop] [Refresh] [Logs]
backend-1	756a0a30671c	testserver:1.0	-	1.77%	30.86K	[Stop] [Refresh] [Logs]
nginx-1	e05031e4d544	testserver:1.0	80:80 (T)	0%	12.37K	[Stop] [Refresh] [Logs]

IV. FLUJO GENERAL DEL SISTEMA

El sensor publica un valor numérico en el topic test01 del broker EMQX. El backend Python recibe el mensaje a través de la suscripción MQTT, lo convierte a número flotante y lo inserta en la base de datos MariaDB. El dashboard del navegador consulta periódicamente el endpoint /api/data a través de Nginx, obtiene las lecturas almacenadas y actualiza la interfaz gráfica en tiempo real.